

Algorithm Design Paradigms

While paradigms are not used heavily in the program, there are two key areas where they are used.

- **Dijkstra's Algorithm - Greedy:** This algorithm is used in the `SimilarItemsGraph` class to find the most similar (shortest path) items to a given item. The algorithm is greedy because it always chooses the next node with the smallest distance (most similar item) to explore, without considering the overall structure of the graph.
 - Why it was chosen
 - One issue of greedy algorithms is that they may not produce the globally optimal solution due to locally optimal choices being chosen always. However, the punishment for this is that paths which go through more nodes are not considered as heavily. This behavior is desirable for the graph in this program. For example, say item A has a similarity score of 5 to items B and C, but the path from A to B is much shorter than the path from A to C. In this case, it is likely that B is more similar to A than C, even though the similarity score is the same, because the path from A to C goes through more items which are not very similar to A.
- **Quick Sort - Divide and Conquer:** This algorithm is used in the `QuickSort` class to sort the table for sizes higher than or equal to 25. The algorithm is a divide and conquer algorithm because it recursively sorts each side of a pivot.
 - Why it was chosen
 - The paradigm here itself did not present with the qualities we wanted. Rather the `QuickSort` algorithm was chosen for its average time complexity of $O(n \log n)$. There is certainly much to improve, particularly with pivot selection, and potentially switching algorithms all together.